

Recipe for Discovery: A Pipeline for Institutional Open Source Activity

Juanita Gomez
jgomez91@ucsc.edu

Emily Lovell
elovell@ucsc.edu

Stephanie Lieggi
slieggi@ucsc.edu

Alvaro A. Cardenas
alacarde@ucsc.edu

James Davis
davis@cs.ucsc.edu

University of California, Santa Cruz
Santa Cruz, California, USA

February 27, 2026

Abstract

Open source software development, particularly within institutions such as universities and research laboratories, is often decentralized and difficult to track. Although academic teams produce many impactful scientific tools, their projects do not always follow consistent open source practices, such as clear licensing, documentation, or community engagement. As a result, these efforts often go unrecognized due to limited visibility and institutional awareness, and the software itself can be difficult to sustain over time.

This paper presents an end-to-end framework for systematically discovering and analyzing open source projects across distributed academic systems. Using ten universities as a case study, we build a pipeline that collects data via GitHub’s REST API, extracts metadata, and predicts both institutional affiliation and project type (e.g., development tools, educational materials, websites, documentation). Applied across the ten campuses, our method identifies over 200,000 repositories and collects information on their activity and open source practices, enabling a deeper understanding of institutional open source contributions.

Beyond discovery, our framework enables actionable insights into institutional open source practices, revealing patterns such as missing licenses or limited community engagement. These findings can guide targeted support, policy development, and strategies to strengthen open source contributions across academic institutions.

1 Introduction

Open source software plays a central role in modern software engineering as the foundation of critical infrastructure and scientific discovery. Despite its impact, understanding where and how open source is produced remains a challenge, particularly when it is developed by decentralized institutions such as universities and research laboratories. In these settings, development is often distributed across accounts with inconsistent practices such as missing licenses or limited documentation. As a re-

sult, many valuable projects go unrecognized and can be difficult to sustain.

Institutions worldwide are increasingly investing in initiatives to support and strengthen their open source ecosystems. Programs such as the Institutional Innovation Grapher at the University of Texas at Austin [1], the OSPO Database Development Playbook at the University of Wisconsin [2], the Stanford Open Source Registry [3], the George Washington University Open Source Project Registry [4], the Janelia OSSI initiative at the Howard Hughes Medical Institute [5], and France’s national research software catalog [6] demonstrate a growing commitment to creating tools that help discover, catalog, and showcase institutional open source projects. By mapping these repositories, institutions can provide support to researchers, improve infrastructure and sustainability, and increase the visibility and impact of their open source contributions.

Unlike software developed within companies or centralized teams, academic open source often originates from small university groups or individual researchers. These projects lack a unified platform, naming convention, or metadata standard, and while GitHub is widely used for collaboration, its vast scale makes discovery and analysis difficult. Institutional information in contributor profiles is often inconsistent or missing, repositories may include forks or class assignments rather than active research software, and contributors frequently use multiple profiles or change affiliations. As a result, simple keyword searches (e.g., “UC Davis”) can return thousands of ambiguous results, making manual filtering impractical.

In this paper, we present a scalable approach for systematically discovering and analyzing academic open source repositories. Our method combines GitHub’s REST API with a filtering pipeline that integrates large language models (LLMs) to infer remove false positives and categorize projects by type. Beyond merely cataloging repositories, the framework allows detailed examination of project characteristics, revealing trends such as absent licenses or inconsistent community standards (e.g.,

contributing guides and templates). These findings highlight opportunities for institutions to provide targeted support, enhancing software sustainability, compliance, and community engagement.

We use the University of California (UC) system as a case study for evaluating our pipeline because it represents one of the world’s largest public research university systems comprising campuses with varying sizes, disciplinary strengths, and levels of open source activity. Applied across the ten UC campuses, our pipeline identifies 236027 repositories. Given the global impact of the UC system and notable open source projects originating from its campuses (e.g., Jupyter from UC Berkeley), our results offer both local and broader insights for strengthening institutional open source practices. Our contributions include:

- We introduce a scalable method for discovering open source repositories affiliated with institutions (including academic and research organizations) by defining a **repository signature** based on repository and contributor metadata.
- We analyze the results of mining the ten UC campuses, producing actionable insights into open source practices, including patterns related to licensing, documentation, and community standards that can guide support and policy.
- We release the code for repository scraping, filtering and analysis as an open source tool, enabling replication and further research. The code is available at: <https://anonymous.4open.science/r/repofinder-E250>.

2 Related Work

There is growing interest in discovering and analyzing open source software repositories to better understand their purpose, usage, and practices. For **discovery**, some universities have developed tools such as the Institutional Innovation Grapher [1], which identifies GitHub accounts that mention a university in their bios. However, this method has limitations, as many developers do not explicitly mention institutional affiliations. Similarly, the University of Wisconsin Playbook [2] aims to discover open source contributions but does not automate affiliation detection. Schwartz et al. [7] present a more comprehensive effort by combining GitHub keyword searches with university website scraping and LLM-based filtering to identify research software engineering (RSE) repositories; however, affiliation is not directly studied.

For **categorization**, previous work has addressed several angles. One of the most common, is distinguishing software engineering projects from other non-research projects. Munaiah et al. [8] implemented a tool called Reaper to classify engineered software projects using a combination of score-based heuristics and random forest classifiers. Pickerill et al. [9] extended this idea with

PHANTOM, a time-series clustering method based on Git logs.

Many efforts focus on categorizing repositories by topic and functionality [10], [11], [12] [13], [14]. Sharma et al.[15] proposed automatic cataloging based on topic modeling of README files, while Zhang et al. [16] used keyword-driven hierarchical classification to assign repositories to topic categories. Aslam et al. [17] built a multilingual source code classifier using machine learning, and Linares-Vasquez et al.[18] classified software by application domain using (API) calls from third-party libraries. Embedding-based models have also been explored: Bharadwaj et al. [19] applied BERT-style models to classify GitHub issues, while Izadi et al. [20] used semantic knowledge graphs to improve topic recommendation.

Regarding **analysis**, prior work has explored repository practices and the sustainability of open source projects. For example, Ehls et al. [21] analyze sources of project failure to identify risks in open source development, while Alami et al. [22] apply sustainability metrics to evaluate how different aspects of open source communities affect software quality. Surveys such as the 2024 Open Source Survey at UW-Madison [23] provide insights into usage patterns and opportunities to improve institutional open source practices.

GitHub mining at scale has also been a focus of previous work. Kalliamvakou et al. [24] highlighted challenges in GitHub data quality and usage patterns. Ma et al. [25] introduced World of Code, an infrastructure for mining global VCS data and exploring interdependencies. Cosentino et al.[26] conducted a systematic mapping study of related GitHub research.

Although these works advance methods for repository discovery, classification, and analysis of open source practices, they generally focus on project characteristics, topic modeling, or development activity, without directly addressing institutional affiliation; or they are limited in scope, focusing on individual projects, small samples, or survey-based data, making it challenging to generalize findings at scale or systematically connect them to institutional contexts.

A recent systematic mapping study [27] surveys 43 works on repository classification and notes that, while many papers cite "discoverability" as a motivation, none explicitly integrates institutional affiliation, project categorization, and actionable analysis at scale. Our work addresses this gap by developing an end-to-end pipeline that systematically identifies institutionally affiliated repositories, classifies them by project type, and analyzes their practices to generate actionable insights. This integrated approach provides institutions with the means to understand and improve their open source ecosystems at scale.

3 Methodology

We now describe our methodology for identifying and characterizing open source projects on GitHub. We first explain how we discover candidate repositories using a

high recall search strategy across repositories, organizations, and users. We then detail the filtering step used to remove false positives. Finally, we describe how the remaining repositories are further categorized by project type to facilitate the analysis of the academic open source landscape.

3.1 Discovery

As the starting point for our data collection, we focus on GitHub because of its dominance in open source development, extensive API support, and widespread adoption. To clarify our data collection process, we first summarize the basic structure of a GitHub repository and the types of metadata available through the GitHub API. A **repository** is the central unit of collaboration on GitHub, containing a codebase, documentation, and configuration files. Each repository includes a number of descriptive **attributes**, such as its name, description, README file, and associated tags. Repositories may also include **community files**, like a code of conduct, contributing guidelines, and security policies, files that tend to signal project maturity and openness to external contributors. **Engagement indicators** such as stars, forks, downloads, and subscriber count reflect how active and visible a project is within the open source ecosystem. Furthermore, each repository is owned by an individual or an **organization** (a special GitHub account for groups), and includes a list of **contributors** who have committed code, often with metadata such as their username and affiliation.

To guide our analysis, we structure the data collection process into three distinct phases. Phase 1: Repository Data, where we collect core metadata and community files from GitHub repositories; Phase 2: Contributor Data, which enriches the dataset with information about individual contributors; and Phase 3: Organization Data which captures additional information from repository owners classified as organizations in GitHub. Figure 1 provides an overview of this multi-phase workflow, highlighting the specific GitHub API endpoints and data elements involved in each stage.

3.1.1 Phase 1: Repository Data

The goal of this phase (illustrated in Figure 1 phase ①) is to identify repositories that might be associated with an institution among those available on GitHub. Because this step deliberately favors recall over precision, we retain all matches and defer removing false positives to the filtering section.

To achieve this, we perform **three distinct but complementary searches** using GitHub’s REST API: (1) a repository search, (2) an organization search, and (3) a user search.

Repository search: We begin by directly querying GitHub’s repository search endpoint (`/search/repositories`) using institution-specific lexical cues. For each University of California (UC) campus, we construct a set of keywords consisting of the university **name**, **acronym**, **email domain**, and a

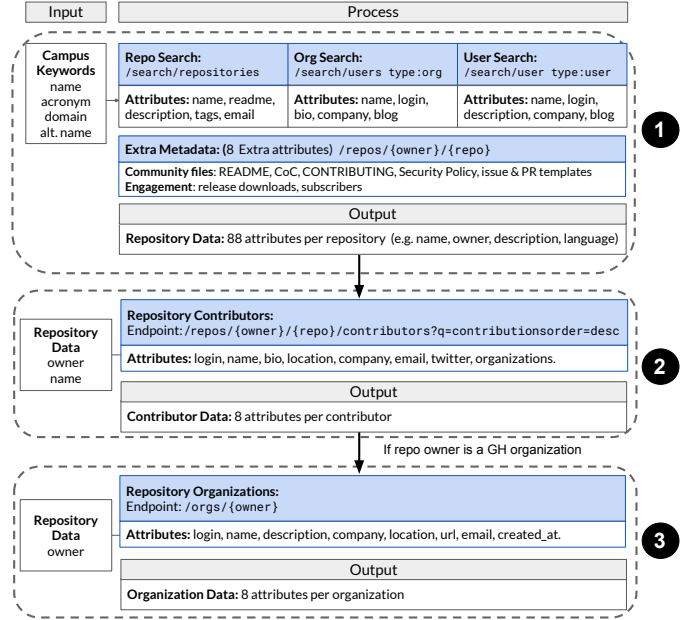


Figure 1: Data collection phases with GitHub’s REST API.

common **alternate name** (e.g., “UC Berkeley”), as shown in Table 1. These keywords are matched against five repository attributes: **name**, **description**, **README**, **tags**, and **email**. For the first four attributes, we query using all keywords, while for the **email** attribute we search only using the university domain.

Organization search: In parallel, we query GitHub’s organization search endpoint (`/search/users` with `type:Organization`) to identify organizations whose profiles suggest institutional affiliation. Using the same set of campus specific keywords, we search across organization **name**, **login**, **description**, **company**, and **blog** fields. Repositories owned by the organizations returned by these queries are collected and added to the candidate repository set.

User search: Finally, we query the user search endpoint (`/search/users` with `type:User`) to identify individual developers whose profiles indicate a possible university affiliation. We search across user **name**, **login**, **bio**, **company**, and **blog** fields using the same institutional keywords. For each matched user, we collect all repositories owned by that user and include them in the dataset.

University Name	Acronym	Domain	Alt. Queries
University of California, Berkeley	UCB	berkeley.edu	UC Berkeley
University of California, Davis	UCD	ucdavis.edu	UC Davis
University of California, Irvine	UCI	uci.edu	UC Irvine
University of California, Los Angeles	UCLA	ucla.edu	UC Los Angeles
University of California, Merced	UCM	ucmerced.edu	UC Merced
University of California, Riverside	UCR	ucr.edu	UC Riverside
University of California, Santa Barbara	UCSB	ucsb.edu	UC Santa Barbara
University of California, Santa Cruz	UCSC	ucsc.edu	UC Santa Cruz
University of California, San Diego	UCSD	ucsd.edu	UC San Diego
University of California, San Francisco	UCSF	ucsf.edu	UC San Francisco

Table 1: Keyword queries.

By combining repositories discovered directly through repository metadata with those obtained indirectly via organization and user ownership, this phase produces a high recall set of candidate repositories potentially associated with UC campuses.

The initial data scraping collects most of the metadata needed for our analysis (80 attributes in total), all based on the values provided by GitHub’s API. We supplement the initial search with additional API requests to collect specific repository features aligned with GitHub’s recommended community standards [28]. These include the presence of key community files, such as a README, code of conduct, contributing guide, security policy, issue and pull request templates, which we search using their most common filenames and paths [29]. Lastly, we collect additional engagement indicators: release downloads and subscriber count. This data helps assess the project’s maturity, community support, and open source practices.

3.1.2 Phase 2: Contributor Data

The second step is to collect information about the contributors to each repository. Having this information is essential for associating the projects with specific universities, since contributor profiles may contain affiliation data that validate a repository’s connection to a specific campus. This helps also filter out some false positives, cases where there are keyword matches in the repository metadata but the repository does not necessarily seem affiliated with the institution. For example, repositories that reference or aggregate content from other (or multiple) universities. We collect the following attributes for each contributor (when available) from repositories identified in Phase 1: `username`, `name`, `bio`, `company` and `email`. The Contributor Data workflow is shown in Figure 1, phase ②.

3.1.3 Phase 3: Organization Data

Finally, we collect additional information about the repository owner *if* it is classified as an organization on GitHub (rather than an individual user). Perhaps the clearest indicator of the affiliation of a repository with a university campus is when the organization that owns it (in GitHub) includes the university domain in its listed website or email address. We refer to these as high-confidence attributes, since they provide direct evidence of institutional affiliation.

We collect data from organizations using the GitHub user API endpoint `/users/owner`, retrieving the attributes: `login`, `name`, `company`, `description`, `email`, `url`. The organization data collection workflow is summarized in Figure 1 phase ③.

3.2 Data Filtering

These three discovery phases yield a large volume of repositories, contributors, and organizations (on the order of hundreds of thousands) spanning all University of California campuses. This broad approach is designed for high recall, but inevitably includes many repositories that are not affiliated with the University of California. For

instance, some repositories mention multiple universities or list academic programs, matching campus-related keywords despite lacking any apparent connection to the UC system. To prepare our dataset for analysis, we first clean the data to remove non informative repositories and then apply a filtering step to estimate the likelihood that each repository is genuinely associated with a university; the following subsections describe these steps in detail.

3.2.1 Data Cleaning

Our initial data cleaning step consists in removing repositories that are forks, archived, templates, or have zero reported size (repositories with no files), as these repositories do not represent original, active development and can introduce noise that affects repository counts and biases analyses of affiliation and activity. All of these attributes were collected during the initial data scraping process.

3.2.2 Exploring Filtering Strategies

Motivated by the broader evolution of classification methods from rule-based and feature-engineered approaches, to embedding-based supervised models, and more recently to large language models (LLMs) capable of contextual reasoning, we evaluated multiple filtering strategies with increasing ability to capture contextual signals.

To compare these approaches we conducted all experiments using data from three UC campuses: UC Santa Cruz (UCSC), UC San Diego (UCSD), and UC Santa Barbara (UCSB). These campuses were selected to reflect institutional diversity within the UC system. Based on undergraduate enrollment sizes [30], UCSD represents a large campus, UCSB a medium-sized campus, and UCSC a smaller campus. Below, we summarize each approach and motivate our final choice. A more detailed description of the alternative methods is available in Appendix A.

Score-Based Classifier (SBC): As a baseline we implemented a rule-based scoring system inspired by prior work in feature engineering [8]. This method assigns heuristic scores to matches between university related keywords and repository attributes (repository metadata, contributor profiles and organization information). While this approach is intuitive and easy to implement, it’s incapable of distinguishing substantive institutional affiliation from superficial keyword matches leading to high false positive rate.

Embedding-Based Machine Learning: To incorporate semantic context beyond exact keyword matches, we explored a supervised learning approach using text embeddings that came from aggregated repository metadata. The embeddings were used as inputs for standard machine learning classifiers (neural networks, random forests, and support vector machines). Although the accuracy of the classification improved, the overhead that the manual labeling of the training set required motivated us to explore alternatives that did not require manual annotation.

Large Language Model Filtering: Our final and most effective approach leverages large language models (LLMs) to assess repositories for university affiliation. We

evaluated multiple OpenAI models: `gpt-3.5`, `gpt-4o`, and `gpt-5-mini`. While all three models were capable of reasoning over repository metadata, `gpt-5-mini` provided the best classification quality and cost-effectiveness.

To evaluate the predictions of the different methods, we manually labeled 100 random data points from each of the 3 campuses. Figure 2 presents a ROC based comparison of these methods. A detailed cost and time comparison is provided in Appendix B

3.2.3 Selected Filtering Method: `gpt-5-mini`

Based on our evaluation of multiple filtering strategies, we adopt large language models (LLMs) as our primary method for determining repository university affiliation and filtering out false positives. We first formalize a definition of what it means for a repository to be affiliated with a university. Then, we use this definition to prompt the model via the OpenAI API with detailed contextual information about each repository, and relevant campus-related keywords.

(1) Defining "Affiliation": Our first step is to define what it means for a repository to be **affiliated** with a university. Informed by our dataset, we identify a **repository’s signature**: Key characteristics that consistently indicate a repository’s affiliation with a university, such as institutional ownership, contributor ties to the campus, and explicit mentions of university work or projects. This allows us to distinguish meaningful connections from incidental keyword matches.

Definition 1. *A repository is considered affiliated with a university if there is clear, public evidence that its development, maintenance, or governance is connected to the university. This includes any of the following criteria:*

1. Research or Academic Unit Development: *Developed or maintained by a research group, lab, center, academic department, or institute that is formally part of the university. Evidence includes README statements such as “developed at ...”, links to lab websites, or institutional pages under the university domain.*

2. Contributor Affiliation (Explicit Evidence): *One or more key contributors (maintainers, lead developers, or primary committers) are affiliated with the university. Evidence includes: (1) university email addresses, (2) public profiles or bios naming the university, or (3) README sections listing team members and their university affiliation.*

3. Institutional or Administrative Development: *Developed or maintained by an official, non-academic university unit (e.g., libraries, Open Source Program Offices, IT departments, or research computing groups).*

4. Official Ownership, Sponsorship, or Endorsement: *Owned, sponsored, or explicitly endorsed by the university. Evidence includes: (1) hosting under an official university GitHub organization, (2) README or website statements indicating university sponsorship or ownership, or (3) copyright notices naming the university.*

5. Documented Collaboration or Partnership: *The README or project website explicitly states a collaboration, partnership, or joint development effort with the university or one of its units.*

6. University-Linked Project Infrastructure: *The project’s official homepage, documentation site, or primary web presence is hosted under the university’s domain or a clearly associated subdomain.*

7. Educational Outreach and Online Courses: *Online learning initiatives affiliated with the university, including: (1) repositories linked to online specializations or courses offered on platforms such as Coursera or edX, and (2) course materials, code examples, or tools developed specifically for such offerings.*

(2) Building the prompt: The prompt asks the model to return a probability value between 0 and 1 indicating the likelihood that the repository is affiliated with the university, along with a brief explanation justifying its assessment. To construct the prompt, we aggregate multiple metadata components: repository metadata $repo_i$, organizational metadata org_i , and information about the top contributors $contrib_i^{(1)}$ and $contrib_i^{(2)}$. Due to the model’s input token limit, each text field is truncated to a maximum of 20000 characters, retaining the first 15000 characters and the final 5000 characters. This truncation primarily affects README files. Based on manual examination of a sample of repositories, we observed that acknowledgements, credits, and contact information that indicate institutional affiliation frequently appear near the end of README files. An example of the constructed prompt is as follows:

You are tasked with determining the likelihood that a GitHub repository belongs to a university based on the following definition:

- Definition: <Definition 1>
- Here is the information about the repository: <repository information>
- And here is the university context: <university information>

Based on this information and the definition above:

- Estimate the probability between 0 and 1 (e.g., 0.87) representing how likely it is that the repository belongs to the university.
- Provide a brief explanation (1-2 sentences) justifying your answer.

Your response must be formatted exactly like this:

- Probability: <value between 0 and 1>
- Explanation: <your explanation here>

3.3 Data Categorization

After identifying repositories affiliated with a university, we further classify them by **project type** to better understand the purpose of institutional open source contributions. This classification enables more granular insight into how universities engage with open source, whether through tool development, education, web hosting, data set creation, or documentation. We designed a set of categories inspired by prior work on repository classification [31], with several adjustments to improve compatibility with LLMs, including the addition of concrete examples and keyword indicators for each category to provide a clearer context and reduce ambiguity.

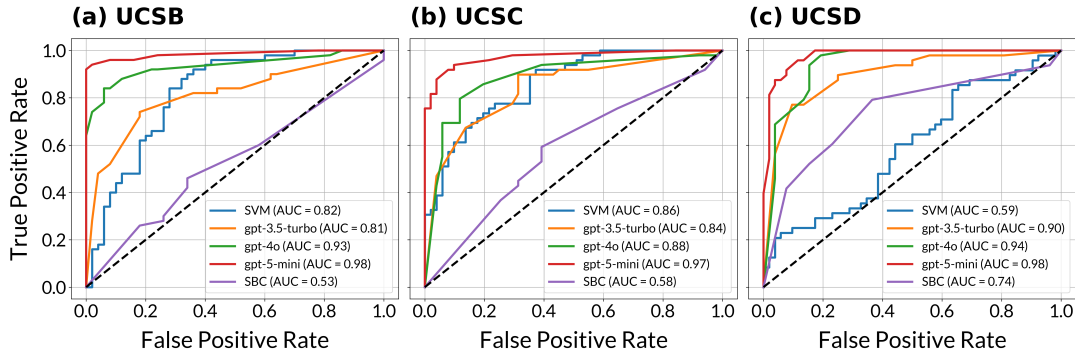


Figure 2: ROC curves for UCSB, UCSC, and UCSD for the five methods evaluated. `gpt-5-mini` achieves the highest area under the curve (AUC) for the three campuses, indicating strong classification performance.

Definition 2. You must classify each GitHub repository into exactly one of the following categories based on its primary purpose. When multiple purposes are present follow the precedence rules described below.

DEV: A repository primarily used for the development and maintenance of a software artifact, including tools, libraries, components, applications, services, or APIs. The presence of documentation, a website, or example code does not override this classification if active software development is the main function.

EDU: A repository primarily used for educational purposes, including course-related materials (e.g., assignments, labs, class projects, or course websites), teaching infrastructure, or instructional content explicitly tied to a course, workshop, or training program. This category includes teaching demos and repositories created to support internships, tutorials, or learning exercises, provided their main goal is instruction rather than production use. If a repository contains software or an application developed as part of completing a course or academic requirement, it is classified as **EDU**, even if it resembles a standalone or production-style software project.

DOCS: A repository primarily used to store or track non-educational documents, such as reports, policies, white papers, specifications, or meeting notes.

WEB: A repository primarily used to host a public-facing website or informational page, such as a project homepage, documentation website, or static or CMS-based informational site. This includes repositories built with static site generators (e.g., Jekyll or Hugo), as well as site-specific styles, layouts, or templates, when the main purpose is presentation rather than software development. Reusable themes or design systems intended for general adoption are excluded and classified as **DEV**. Personal websites, portfolios, or “about me” pages are classified as **OTHER**.

DATA: A repository primarily used to store, curate, or distribute datasets, including research datasets, benchmarks, or data collections (including images). Lightweight scripts for data loading or inspection do not change this classification.

OTHER: A repository that does not clearly align with any of the above categories, such as empty repositories, personal experiments, configuration-only repositories, or miscellaneous content without a clear primary purpose.

The classification prompt provides the LLM with a standardized set of definitions and relevant repository

metadata: full name, description, README content, number of stars, forks, and contributors. The model is then asked to output the best fit category along with a brief explanation. The prompt used is shown below:

```
You must classify each GitHub repository into exactly
one of the following categories: <Definition 2>
Choose the most appropriate category based on the
repository information.

• Here is the information about the repository:
  <repository information>

• Your task: Return only the predicted category
  (one of: DEV, EDU, DOCS, WEB, DATA, OTHER), and a
  short explanation.

• Your response must be formatted exactly as:
  - Category: <one of the 6 categories>
  - Explanation: <brief explanation>
```

4 Results

4.1 Discovery

Table 2 illustrates the breakdown of the repositories collected on the ten campuses of the University of California (UC). We found 236037 unique repositories with our preliminary collection, and given the scale, manual verification and filtering of each repository is impractical.

University	Repos	Contributors	Organizations
UCB	36552	34981	1108
UCD	26549	16400	695
UCI	28730	20748	657
UCLA	24917	18397	651
UCM	10132	9405	392
UCR	16050	10492	432
UCSB	19907	12051	651
UCSC	24181	14377	662
UCSD	42729	24981	908
UCSF	6290	4731	373
Total	236037	166563	6529

Table 2: Repository and contributor statistics by UC campus.

4.2 Filtering

We now present the results of the filtering procedure for three universities with available test data (UCSD, UCSC, and UCSB) evaluating the pipeline’s ability to correctly identify institutionally affiliated repositories and filter out false positives.

4.2.1 Accuracy Analysis

As discussed in Section 3.2, **gpt-5-mini** achieved the best performance at filtering out false positives. We determine the optimal classification threshold for this model by maximizing Youden’s J statistic [32], and we evaluate its performance using standard metrics, including accuracy and the F1 score. The results are summarized in Table 3.

University	F1-score	Accuracy	False Pos. (%)	False Neg. (%)
UCSB	0.90	0.91	0%	9%
UCSC	0.90	0.91	1%	8%
UCSD	0.93	0.94	2%	4%

Table 3: Filtering performance across three UC campuses for **gpt-5-mini**.

As shown in Table 3, both accuracy and F1 score exceed 90% across all three campuses, indicating that **gpt-5-mini** performs well at identifying university-affiliated repositories. The majority of classification errors are false negatives, repositories that should be marked as affiliated but are missed by the language model. Representative examples include R[35], R[36], R[37], R[38], and R[39]. In many of these cases, the university connection is subtle, such as a faculty member being mentioned only at the end of a README file or brief, single-line acknowledgments that require careful attention. The language model appears to struggle particularly when repositories have contributors from multiple institutions, making the institutional affiliation less explicit. These observations suggest that when affiliation evidence is difficult to locate, language models are more likely to fail to identify it.

A smaller number of false positives were also observed. For example, the repository R[40] contains multiple links using a university domain and hosts scripts and data; however, it lacks a clear or substantive connection to the university itself.

4.2.2 Affiliated Repositories Per Campus

We use the **gpt-5-mini** model to filter false positives from the repositories of the remaining seven campuses, yielding the results shown in Table 4. The table presents the total number of repositories, the number remaining after data cleaning, and the number of repositories classified as affiliated.

Overall, only about half of the cleaned repositories are predicted to be truly affiliated with a UC campus, highlighting the substantial amount of noise present in the initial dataset. A significant portion of projects, ap-

University	Repos	Cleaned	# Affil.	% Affil.
UCB	36552	25810	12232	47.4%
UCD	26549	20526	8736	42.6%
UCI	28730	21958	8134	37.0%
UCLA	24917	18088	9859	54.5%
UCM	10132	8395	1989	23.7%
UCR	16050	12603	5672	45.0%
UCSB	19907	14679	7926	54.0%
UCSC	24181	18266	6264	34.3%
UCSD	42729	28416	18163	63.9%
UCSF	6290	4537	2665	58.7%
Total	236037	173278	81640	47.1%

Table 4: Predicted affiliated repositories per university using **gpt-5-mini**.

proximately 27% of the original 236K repositories were excluded during the cleaning stage because they corresponded to templates, forks, archived repositories, or empty projects, reflecting the need for careful preprocessing when working with this large scale repository data. Affiliation rates vary considerably across campuses: while UCSD, UCLA, UCSB, and UCSF show affiliation rates above 50%, UCM exhibits a notably lower rate (23.7%). This lower affiliation rate is likely influenced by the relatively small size of the campus. As one of the smallest institutions in the UC system, UC Merced has less research contributions, which results in fewer affiliated open source projects overall. San Diego (UCSD) stands out as the campus with the largest absolute number of affiliated repositories, which is consistent with its scale and research intensity as one of the largest research universities in the system.

4.3 Project Type Classification

We now present the results of the project type categorization, which identifies the main purpose of each affiliated repository using predefined category labels.

4.3.1 Accuracy Analysis

Given the overall better performance of **gpt-5-mini** in the previous section, we select this model for project type classification. We first evaluate its performance on three campuses (UCSB, UCSC, and UCSD), for which we manually label 100 repositories per campus, sampled from the set of repositories previously predicted as affiliated. These manual labels are used to assess the accuracy of the GPT-based predictions. We do not balance the sample by project type; instead, we preserve the natural distribution of project types to evaluate whether the model’s predicted class distribution aligns with the underlying distribution in the data.

We report results using accuracy per project type, calculated against the manually labeled test set. Table 5 shows the model’s performance across the three universities, along with the number of repositories labeled for each project type in the test subsets.

Overall, **gpt-5-mini** achieves consistently high accuracy across the three universities (90–92%), compara-

Project Type	UCSB		UCSC		UCSD	
	Count	Acc.	Count	Acc.	Count	Acc.
DATA	8	0.62	0	0.00	4	0.50
DEV	26	0.92	30	0.90	21	0.86
DOCS	0	0.00	3	0.33	1	0.00
EDU	51	0.98	50	0.96	65	0.98
OTHER	12	0.92	9	0.89	6	0.83
WEB	3	0.67	8	0.75	3	1.00
Total	100	0.92	100	0.90	100	0.92

Table 5: Project type classification results per university. Each cell shows the number of repos and accuracy.

ble to the performance observed for affiliation prediction, suggesting that the model generalizes well to this downstream classification task. The highest accuracies are observed for EDU and DEV repositories, which is expected given the presence of strong and explicit indicators of purpose in these projects, such as course identifiers, assignment structures, or deployment instructions. In contrast, DATA and WEB repositories are more challenging to classify accurately without explicit metadata about the data types within the repositories, and their README files or descriptions can be ambiguous, sometimes resembling documentation or development repositories. DOCS repositories are relatively rare in the test sets and exhibit lower accuracy, reflecting their inherent ambiguity and overlap with WEB and DEV categories, particularly when documentation is colocated with code or hosted as a static site.

4.3.2 Distribution of Repositories by Project Type

We use `gpt-5-mini` to classify the affiliated repositories for all 10 universities and analyze their distribution across the predefined repository types for each UC campus, as summarized in Table 6.

Overall, the distribution is dominated by DEV and EDU repositories, which together account for approximately 85% of all affiliated projects (42.9% DEV and 42.1% EDU). This balance suggests that GitHub serves both as a platform for active software development and as a central infrastructure for instructional use across the UC system. Development repositories are particularly prevalent at research-intensive campuses such as UCB and UCLA, where nearly half of affiliated projects fall into the DEV category.

Educational repositories constitute a substantial share at most campuses, with particularly high concentrations at UCSB (60.3%), UCR (46.6%), and UCM (45.3%), reflecting the strong presence of course related materials, assignments, and student projects. In contrast, UCSF exhibits a different distribution: 63.0% of its affiliated repositories are classified as DEV, while only 18.3% fall into the EDU category. This pattern is consistent with UCSF’s role as a graduate only focused on health sciences. Since UCSF does not offer undergraduate degrees, it has a smaller pool of students enrolled in coursework, leading to fewer educational repositories and a stronger empha-

sis on research-oriented projects. Across all campuses, WEB and DATA repositories each represent a relatively small fraction of affiliated projects (typically 2–5%), while DOCS repositories are rare, consistently accounting for around 1% or less.

5 Insights on Affiliated Repositories

With these predictions, we can gain insight into the types of practices and trends of university-affiliated repositories. We start by analyzing the programming languages and software licenses used in the affiliated repositories.

5.1 Programming Language Distribution

Figure 3 shows the aggregated distribution of programming languages in all affiliated repositories from the 10 UC campuses, broken down by project type.

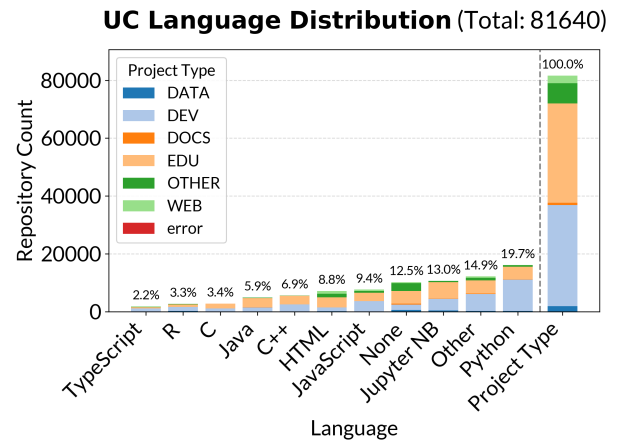


Figure 3: Programming language distribution across all ten University of California campuses. Languages representing less than 2% of total usage were aggregated under “Other.” The bar on the right labeled “Project type” shows the overall distribution of repository types and serves as a baseline for interpreting language usage within each category.

Python stands out as the language most commonly used in general, aligning with GitHub’s 2024 Octoverse report [33], which also ranked Python as the top language in public repositories. Although Python is the most dominant language within the development (DEV) repositories, it still represents only 17.4% of all repositories, reflecting the general diversity in language usage. Jupyter Notebook is also highly prevalent, especially in educational (EDU) repositories, where it serves as a popular format for teaching and interactive coursework. HTML is the most frequently used language in web-related (WEB) repositories, closely followed by JavaScript. Other notable languages include Java, mainly used in educational projects, C++, C, and R which appear across both DEV and EDU types.

5.2 License Distribution

Beyond language usage, understanding the licensing choices of repositories provides insight into how openly

University	DEV	EDU	DOCS	WEB	DATA	OTHER	Total
UCB	47.0%	38.7%	0.9%	3.2%	2.1%	8.1%	12232
UCD	44.0%	39.5%	1.2%	3.1%	3.1%	9.1%	8736
UCI	49.7%	33.5%	0.7%	3.0%	1.6%	11.4%	8134
UCLA	48.2%	36.4%	1.2%	3.6%	2.5%	8.2%	9859
UCM	38.3%	45.3%	0.8%	3.4%	3.2%	8.9%	1989
UCR	37.4%	46.6%	0.5%	4.0%	1.7%	9.9%	5672
UCSB	27.6%	60.3%	0.8%	2.9%	2.5%	6.0%	7926
UCSC	42.3%	43.3%	0.8%	2.8%	1.9%	8.9%	6264
UCSD	39.9%	46.0%	0.8%	2.9%	2.0%	8.4%	18163
UCSF	63.0%	18.3%	1.9%	4.8%	4.5%	7.5%	2665
TOTAL	42.9%	42.1%	0.9%	3.2%	2.3%	8.6%	81640

Table 6: Percentage distribution of project types by university, with total affiliated repository counts.

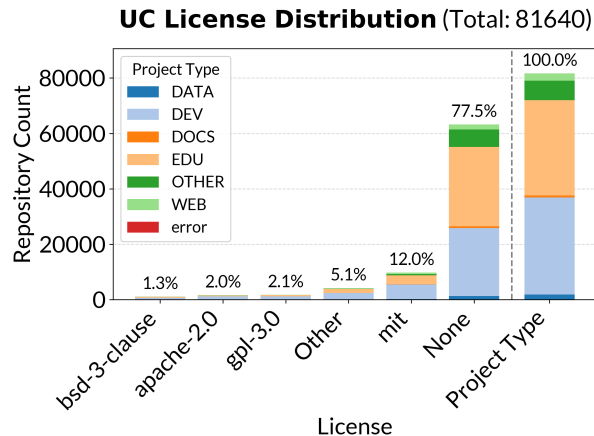


Figure 4: Software License usage across all 10 UCs. Licenses representing less than 1% of total usage were aggregated under “Other.” The bar on the right labeled “Project type” shows the overall distribution of repository types and serves as a baseline for interpreting distribution within each type.

these projects are intended to be shared and reused. Figure 4 shows the distribution of software licenses from the 10 UCs. A notable trend is that over 70% of repositories appear to lack an explicit license. It is important to note that “no license” does not necessarily mean a repository has no license; in some cases, the GitHub API is simply unable to retrieve it. Many of these unlicensed repositories are development (DEV) projects, likely smaller research efforts or student works not intended for broad open source distribution. Almost 100% of the repositories in the category ‘OTHER’ have no license, consistent with their varied and often informal content. Among repositories that specify a license, MIT is by far the most popular, covering 12% of all repositories, representing more than one third of those that have any license, and is prevalent across both educational (EDU) and development projects (DEV), as well as a smaller portion of web projects. There is also notable license diversity beyond MIT, including GPL, Apache, and BSD licenses. At least 5% use a variety of other licenses, each representing less than 1% individually, suggesting a wide range of license choices across

the dataset.

5.3 Adoption of GitHub Community Standards

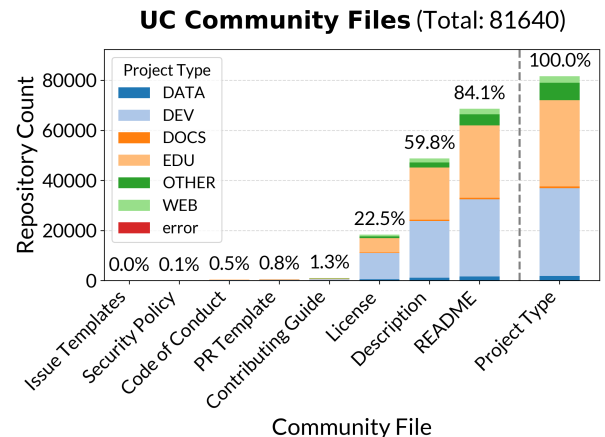


Figure 5: Presence of community files across all ten University of California campuses. The bar on the right labeled “Project type” shows the overall distribution of repository types and serves as a baseline for interpreting community file usage within each category.

We also aim to understand how people developing software within academic contexts engage with community practices. To do so, we analyze the presence of the community standards recommended by GitHub as indicators of openness, collaboration, and project sustainability.

Figure 5 illustrates the percentage of repositories that include GitHub’s recommended community standards [28]: Description, README, License, Code of Conduct, Contributing Guide, Security Policy, Issue Templates, and Pull Request Templates. Including these files is considered a best practice in open source development, as they help make a project easier to understand, use, and contribute.

Most repositories include a README file (around 85%) and a description (around 60%), making these the most commonly adopted community practices. Only about 23% per university have an explicit license, but when looking at the other community guidelines, the percentages reduce considerably. Less than 2% of the repos-

itories provide contributing guidelines, codes of conduct, security policies or templates for issues and pull requests. This trend suggests that most of the repositories are small scale efforts, rather than mature open source tools that expect outside contributions. It may also reflect a general lack of awareness among students and researchers about open source best practices, even though they are essential to enabling collaboration and long term sustainability.

We are particularly interested in the repositories classified as development (DEV), as these are the most likely to be intended for collaborative software development. Additionally, we want to understand how project popularity influences the adoption of community standards under the assumption that more popular projects may attract more contributors, and therefore require better documentation and engagement practices. We use the number of stars as a measure of popularity since they provide a useful indication of interest and community engagement, and we focus our analysis on DEV repositories across all UC campuses. Figure 6 presents a heatmap showing the presence of community files in the DEV repositories, segmented by popularity. We group repositories into five star buckets based on their number of GitHub stars and compute the percentage of repositories in each bucket that include each community file.

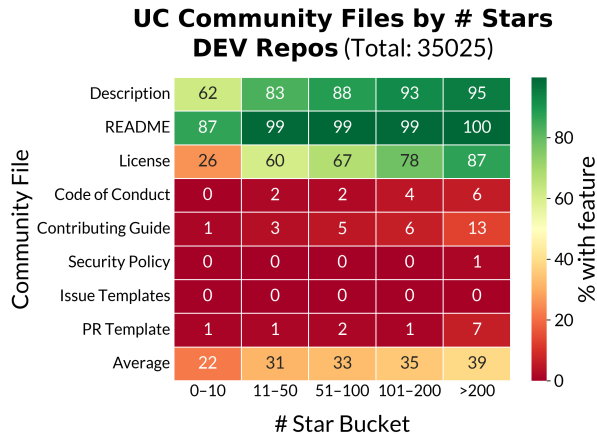


Figure 6: Presence of community files across all 10 UC campuses of DEV repositories by number of stars

As shown in the heatmap, most DEV repositories include a description and README file; however, these practices become nearly universal among more popular projects. Specifically, more than 90% of repositories with more than 100 stars include a description and a README. Licensing shows a particularly strong correlation with popularity: almost 80% of repositories with 50–100 stars declare a license, and this increases to 87% for those with more than 200 stars. A similar upward trend is observed for the presence of a code of conduct and contributing guide, both of which increase monotonically across star buckets. However, their overall adoption remains limited: only 6% of the projects include a code of conduct and 14% include a contributing guide, even in

the highest popularity tier. For pull request templates, the trend is less evident, with only a big rise in adoption once projects exceed 200 stars. Finally, security policies and issue templates remain rare across the board.

To further explore the relationship between project popularity and the adoption of community practices, we compute the average presence of community files across DEV repositories, grouped by star count. For this, we divide repositories into 20 evenly spaced buckets based on their number of stars, capping the maximum at 1000 to ensure a balanced scale (the most starred project in our dataset has over 40000 stars). For each bucket, we calculate the percentage of repositories that include each community file and then average these percentages across all community file types.

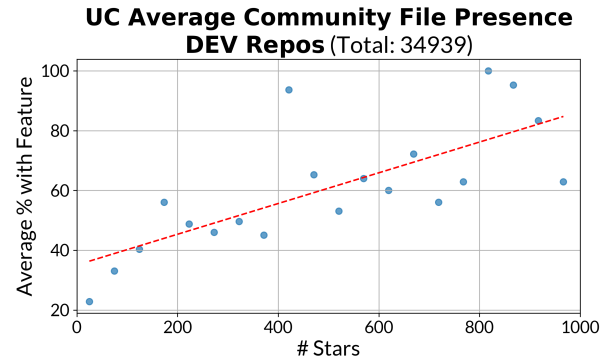


Figure 7: Average presence of community files in DEV repositories across all 10 UC campuses, grouped by GitHub star count (capped at 1000 stars).

The resulting scatter plot (Figure 7) shows a clear positive correlation: As the number of stars increases, so does the average presence of community files. Despite this correlation, the overall adoption of community standards remains relatively low. There are projects with more than 200 stars showing less than 50% adoption, highlighting that many best practices of collaborative development are still underutilized throughout the UC ecosystem. There is a clear gap and an opportunity for improvement: increasing awareness and helping project maintainers navigate these practices could strengthen the open source culture within academic settings.

6 Conclusions and Future Work

In this work, we introduced a framework for identifying, filtering, and analyzing open source repositories affiliated with decentralized institutions. Our results reveal that many repositories initially retrieved are not truly affiliated with the studied institutions, and among those that are, most are educational (EDU) rather than development (DEV) projects. This distribution underscores both the diversity of institutional repositories and the necessity of automated filtering methods to enable meaningful and scalable analysis.

Our study allowed us to obtain a comprehensive picture of the open source landscape of the UC system. This

analysis provides insights into community practices across affiliated repositories, identifying key gaps and opportunities where institutions, such as the Open Source Program Offices (OSPOs) can provide additional support and resources to strengthen these communities.

Although our case study focuses on the University of California (UC) system, the proposed pipeline is not specific to any single university system and can be applied to a wide range of institutions. Researchers can easily adapt it to different institutions by modifying a single configuration file containing institution specific keywords (e.g., names, acronyms, and domains). Once configured, the pipeline automatically collects, classifies, and analyzes affiliated repositories. By openly sharing our code, we aim to enable other institutions to map and better understand their open-source ecosystems.

As a next step, we will build a public browser of open source projects across the UC university systems, enabled by the fact that our database allows us to systematically identify high impact projects using engagement and popularity indicators such as stars, forks, and contributor activity. This capability makes it possible to move beyond discovery and toward prioritizing projects that are actively used and maintained.

This centralized browser will showcase the contributions of universities to open source scientific projects while also helping to surface missing or inconsistent practices and supporting maintainers in strengthening them. Notably, many institutions have already attempted to track and promote impactful open-source projects by building project registries but this work is typically performed manually and does not scale. Our approach provides an automated, extensible alternative that enables institutions to more effectively support their open source communities.

References

- [1] University of Texas OSPO. Institutional Innovation Grapher. <https://github.com/UT-OSPO/institutional-innovation-grapher>, 2023. Accessed: 2025-06-18.
- [2] Simon Goring. University of wisconsin open source monitoring project. <https://simongoring-ospo.curve.space/>, 2023. Accessed: 2025-06-18.
- [3] OpenSource@Stanford. Opensource@stanford projects registry. <https://opensource.stanford.edu/projects-registry>, 2025. Accessed: 2025-06-19.
- [4] The George Washington University Open Source Program Office. Project registry, 2025. URL <https://ospo.gwu.edu/project-registry>. Accessed: 2025-10-19.
- [5] Open Source Science Initiative at Janelia. Janelia ossi software projects. <https://ossi.janelia.org/projects/>, 2025. Accessed: 2025-06-19.
- [6] Ministère de l'Éducation nationale, de l'Enseignement supérieur et de la Recherche. Catalogue des logiciels enseignement supérieur and recherche. <https://logiciels.catalogue-esr.fr/>, 2025. Accessed: 2025-06-19.
- [7] Samuel David Schwartz, Boyana Norris, and Stephen Fickas. An empirical survey of github repositories at u.s. research universities, August 2024. URL <https://doi.org/10.5281/zenodo.13368204>.
- [8] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. Curating github for engineered software projects. *Empirical Software Engineering*, 22(6): 3219–3253, 2017. doi: 10.1007/s10664-017-9512-6. URL <https://doi.org/10.1007/s10664-017-9512-6>.
- [9] Patrick Pickerill, Henrik J. Jungen, Mirosław Ochodek, and Meiyappan Nagappan. PHANTOM: Curating GitHub for engineered software projects using time-series clustering. *Empirical Software Engineering*, 25(4):2897–2929, 2020. doi: 10.1007/s10664-020-09825-8. URL <https://doi.org/10.1007/s10664-020-09825-8>.
- [10] Juri Di Rocco, Davide Di Ruscio, Claudio Di Sipio, Phuong Nguyen, and Riccardo Rubei. Topfilter: An approach to recommend relevant github topics. In *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, ESEM '20, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450375801. doi: 10.1145/3382494.3410690. URL <https://doi.org/10.1145/3382494.3410690>.
- [11] Claudio Di Sipio, Riccardo Rubei, Davide Di Ruscio, and Phuong T. Nguyen. A multinomial naïve bayesian (mnb) network to automatically recommend topics for github repositories. In *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering*, EASE '20, page 71–80, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450377317. doi: 10.1145/3383219.3383227. URL <https://doi.org/10.1145/3383219.3383227>.
- [12] Tao Wang, Gang Yin, Xiang Li, and Huaimin Wang. Labeled topic detection of open source software from mining mass textual project profiles. In *Proceedings of the First International Workshop on Software Mining*, SoftwareMining '12, page 17–24, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450315609. doi: 10.1145/2384416.2384419. URL <https://doi.org/10.1145/2384416.2384419>.
- [13] Ratnadira Widyasari, Zhipeng Zhao, Thanh Le Cong, Hong Jin Kang, and David Lo. Topic recommendation for github repositories: How far can extreme multi-label learning go? In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 167–178, 2023. doi: 10.1109/SANER56733.2023.00025.
- [14] Maliheh Izadi, Abbas Heydarnoori, and Georgios Gousios. Topic recommendation for software repositories using multi-label classification algorithms, 2021. URL <https://arxiv.org/abs/2010.09116>.
- [15] Abhishek Sharma, Ferdian Thung, Pavneet Singh Kochhar, Agus Sulistya, and David Lo. Cataloging github repositories. In *Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering*, pages 314–319, 2017. doi: 10.1145/3084226.3084287. URL <https://doi.org/10.1145/3084226.3084287>.

- [16] Yu Zhang, Frank F. Xu, Sha Li, Yu Meng, Xuan Wang, Qi Li, and Jiawei Han. Higitclass: Keyword-driven hierarchical classification of github repositories, 2021. URL <https://arxiv.org/abs/1910.07115>.
- [17] Muhammad Muneeb Aslam, Muhammad Farhan, Sana Yaseen, Ahmad Raza, Muhammad Javed Iqbal, and Muhammad Munwar Iqbal. A smart model for categorization of github repositories. *KIET Journal of Computing and Information Sciences*, 6(1):50–64, 2022. doi: 10.51153/kjicis.v6i1.149. URL <https://kjicis.kiet.edu.pk/index.php/kjicis/article/view/149>.
- [18] Mario Linares-Vásquez, Collin McMillan, Denys Poshyvanyk, and Jairo Aponte. On using machine learning to automatically classify software applications into domain categories. *Empirical Software Engineering*, 19(3):582–618, 2014. doi: 10.1007/s10664-012-9230-z. URL <https://doi.org/10.1007/s10664-012-9230-z>.
- [19] Shikhar Bharadwaj and Tushar Kadam. Github issue classification using bert-style models. In *2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 40–43, 2022. doi: 10.1145/3528588.3528663.
- [20] Mohammad Izadi, Mehran Nejati, and Abbas Heydarnoori. Semantically-enhanced topic recommendation systems for software projects. *Empirical Software Engineering*, 28(50):1–37, 2023. doi: 10.1007/s10664-022-10272-w. URL <https://doi.org/10.1007/s10664-022-10272-w>.
- [21] Daniel Ehls. Open source project collapse – sources and patterns of failure. In *Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS)*, 2017. URL https://aisel.aisnet.org/hicss-50/os/open_source/3/.
- [22] Adam Alami, Raúl Pardo, and Johan Linåker. Free open source communities sustainability: Does it make a difference in software quality? *Empirical Software Engineering*, 29(6):1–30, 2024. URL <https://doi.org/10.1007/s10664-024-10529-6>.
- [23] University of Wisconsin–Madison Open Source Program Office. 2024 open source survey results, 2024. URL https://uw-madison-dsi.github.io/open_source_survey_results/. Accessed: 2025-10-19.
- [24] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining github. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, pages 92–101, 2014. doi: 10.1145/2597073.2597074. URL <https://doi.org/10.1145/2597073.2597074>.
- [25] Yuxing Ma, Chris Bogart, Sadika Amreen, Russell Zaretzki, and Audris Mockus. World of code: An infrastructure for mining the universe of open source vcs data. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, pages 143–154, 2019. doi: 10.1109/MSR.2019.00031.
- [26] Valerio Cosentino, Javier L. Cánovas Izquierdo, and Jordi Cabot. A systematic mapping study of software development with github. *IEEE Access*, 5:7173–7192, 2017. doi: 10.1109/ACCESS.2017.2682323.
- [27] Stefano Balla, Thomas Degueule, Romain Robbes, Jean-Rémy Falleri, and Stefano Zacchiroli. Automatic Classification of Software Repositories: a Systematic Mapping Study. In *International Conference on Evaluation and Assessment in Software Engineering (EASE 2025)*, Istanbul, Turkey, June 2025. URL <https://hal.science/hal-05049757>.
- [28] GitHub. Github community guidelines. <https://docs.github.com/en/site-policy/github-terms/github-community-guidelines>, 2024. Accessed: 2025-06-11.
- [29] GitHub. Building communities documentation. <https://docs.github.com/en/communities>, 2025. Accessed: 2025-10-19.
- [30] Fall enrollment at a glance, university of california. <https://www.universityofcalifornia.edu/about-us/information-center/fall-enrollment-glance>, 2024. Accessed: 2025-06-12.
- [31] Marcus Soll and Malte Vosgerau. Classifyhub: An algorithm to classify github repositories. In Gabriele Kern-Isberner, Johannes Fürnkranz, and Matthias Thimm, editors, *KI 2017: Advances in Artificial Intelligence*, pages 373–379, Cham, 2017. Springer International Publishing. ISBN 978-3-319-67190-1.
- [32] Daniel Berrar. Performance measures for binary classification. In Shoba Ranganathan, Michael Gribskov, Kenta Nakai, and Christian Schönbach, editors, *Encyclopedia of Bioinformatics and Computational Biology*, pages 546–560. Academic Press, Oxford, 2019. ISBN 978-0-12-811432-2. doi: <https://doi.org/10.1016/B978-0-12-809633-8.20351-8>. URL <https://www.sciencedirect.com/science/article/pii/B9780128096338203518>.
- [33] GitHub Staff. Octoverse: Ai leads python to top language as the number of global developers surges. <https://github.blog/news-insights/octoverse/octoverse-2024/>, October 2024. Updated November 22, 2024.
- [34] OpenAI. Openai platform documentation: Embeddings, 2024. URL <https://platform.openai.com/docs/guides/embeddings>. Accessed: 2025-06-12.

Repositories

- R[35] <https://github.com/quantumlib/OpenFermion>, .
- R[36] <https://github.com/quantumlib/OpenFermion-PySCF>, .
- R[37] <https://github.com/PacktPublishing/Hands-on-Design-Patterns-and-Best-Practices-with-Julia>.
- R[38] <https://github.com/irenelopez42/C-specialization>.
- R[39] https://github.com/pradeepsk13/vsdsram_sky130_1.8V.

A Alternative Filtering Methods

In addition to the approach presented in the main text, we explored several alternative filtering methods. While these methods provided useful insights, they were ultimately not used in our final analysis due to lower performance or practical limitations.

A.0.1 Method 1: Score Based Classifier (SBC)

Our baseline system is inspired by previous work on feature engineering; where a feature of the repository, such as finding a keyword associated with a university in the metadata, is assigned a score. The intuition is that whenever we detect a feature contributing to the repository’s signature of affiliation, we increase the probability that the SBC model labels it as a positive case.

In particular, to estimate the probability p_i that a repository r_i is affiliated with a university, we apply a rule-based scoring system consisting of three main steps. First, we perform a keyword search across multiple data attributes. This includes scanning the repository’s metadata component repo_i including its name, description, homepage, and README, for campus-related terms from Table 1. We also examine the organization metadata org_i . If the owner is an organization, we inspect its attributes: name, description, email, URL and company for matches. Additionally, we analyze the top two contributors $\text{contrib}_i^{(1)}$ and $\text{contrib}_i^{(2)}$, examining attributes such as email, name, bio, and company to determine potential university affiliation. Each attribute a has an associated probability score $s_a \in [0, 1]$ representing how likely a match in that attribute indicates university affiliation. Let:

- $m_{i,a} \in \{0, 1\}$ indicate whether repository r_i has a match in attribute a ,
- p_i be the total probability estimate for repository r_i .

The total probability is computed as the sum of individual attribute probabilities, capped at 1:

$$p_i = \min \left(\sum_{a \in \text{repo}_i} s_a m_{i,a} + \sum_{a \in \text{org}_i} s_a m_{i,a} + \sum_{j=1}^2 \sum_{a \in \text{contrib}_i^{(j)}} s_a m_{i,a}, 1 \right)$$

The specific scores are heuristically tuned using high-confidence attributes and are detailed in Table 7.

A.0.2 Method 2: Text Embeddings & Machine Learning

Instead of relying only on keyword matches, our next approach creates a feature vector by using an embedding with the repository’s signature, and then uses that feature vector as an input to a machine-learning classifier. Notice that the embedding captures the broader context

Metadata	Attribute Checked (a)	Matched Criteria	Probability Score (s_a)
repo_i	homepage	University domain	1.0
	readme, description, name	University name, acronym, or alt. queries	0.20 per match per attribute
org_i	url	University domain	1.0
	email	University domain	1.0
	name, description, company	University name, acronym, or alt. queries	0.30 per match per attribute
$\text{contrib}_i^{(j)}$	email, name, bio, company	University domain	0.50 per match per attribute
	email, name, bio, company	University name, acronym, or alt. queries	0.20 per match per attribute

Table 7: Scoring system for estimating university affiliation.

in which university-related terms appear. For example, the phrase “developed by UC Santa Cruz” carries much stronger affiliation than an incidental mention of the campus name.

Embedding: We aggregate the repository attributes into a single text block. For each repository r_i , we define:

$\mathbf{m}_i = (\text{repo}_i, \text{org}_i, \text{contrib}_i^{(1)}, \text{contrib}_i^{(2)})$, where each component is a set of textual fields. We concatenate these fields into a single raw text string $t_i \in \mathbb{T}$, such that $t_i = \text{concat}(\mathbf{m}_i)$. We finally transform each t_i into a vector representation $\mathbf{v}_i \in \mathbb{R}^d$ using OpenAI’s text embedding model [34] $\mathcal{E} : \mathbb{T} \rightarrow \mathbb{R}^d$. Given the input size constraints of the small embedding model, we truncate each text field to a maximum of 20,000 characters. Since the README is typically the longest field, this truncation primarily affects its content.

Training: We randomly sample 200 repositories and manually assign a binary label to each one, $y_i \in \{0, 1\}$, where $y_i = 1$ indicates university affiliation. We intentionally avoid balancing the dataset to preserve the natural class distribution.

We tested several machine learning models, including neural networks, random forests, and Support Vector Machines (SVM). The classification results were very similar among all models, but SVMs gave slightly better results, so we showcase them for the rest of this paper. After training, for each repository, the model outputs a probability per class: $p_i = f_{\text{prob}}(\mathbf{v}_i)$, where p_i is the predicted probability that r_i is affiliated with the target university.

This approach requires a manually labeled training dataset, which introduces significant overhead in terms of human effort and time.

B Cost and Runtime Comparison

Given the complexity of detecting affiliated repositories, it is also important to consider the practical trade offs in

Model	Avg. Chars	Input Tokens	Output Tokens	Price per 1M Tokens (I/O)	Cost per Repo	Cost × 236K	Time (min)
SBC	—	—	—	\$0 / \$0	\$0	\$0	18
SVM	2,900	725	—	\$0.00002 / —	\$0.0000145	\$3.42	1022
gpt-3.5	4,500	1,125	100	\$0.0005 / \$0.0015	\$0.00056 / \$0.00015	\$167.6	3708
gpt-4o	4,500	1,125	100	\$2.50 / \$10.00	\$0.00381	\$899.75	4330
gpt-5-mini	4,500	1,125	100	\$0.25 / \$2.00	\$0.00048	\$113.46	30342

Table 8: Cost and average time comparison for 236,000 inputs using OpenAI models, based on average input size and token pricing. Single-threaded time is used for gpt-5-mini.

cost and runtime across different modeling approaches. Table shows a cost comparison for different methods of processing GitHub repository data. For the SVM approach the cost reflects transforming the text into vector embeddings. The total time is computed by adding the embeddings time to the model training and prediction time, which in most cases is around 0.70 seconds. For the other methods, costs are based on using the OpenAI API with prompts whose length includes both dynamic repository data and a static prompt component. We compute average input sizes by concatenating all relevant data fields, resulting in approximately 2,900 characters of dynamic text plus about 1,600 characters of static prompt text for the API models. The cost estimates are scaled to 236K repositories, which corresponds to the total number of repositories found for the UC System. The time is computed for `gpt-4o` and `gpt-3.5`. For `gpt-5-mini`, which is much slower, we report the single-threaded runtime estimated from multithreaded benchmarks.

While `gpt-5-mini` achieves competitive classification accuracy and is relatively inexpensive compared to `gpt-4o`, its runtime presents a significant practical limitation for large-scale datasets. As shown in Table B, processing 236,000 repositories with `gpt-5-mini` costs just over \$100, making it a cost-effective option. However, the single-threaded runtime is estimated to exceed 21 days, reflecting its much slower processing speed compared to `gpt-4o` or `gpt-3.5`. These results illustrate a key tradeoff between cost, accuracy, and efficiency, emphasizing that model selection should consider not only predictive performance but also practical constraints such as dataset size and processing time.